Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Основы машинного обучения»
Тема: «Знакомство с анализом данных: предварительная обработка и визуализация»

Выполнил:
Студент 2-го курса
Группы АС-68
Козляковская М.В.
Проверил:
Крошенко А.А.

Брест 2026

Цель: Изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.
Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.
Вариант 3
Регрессия (Прогнозирование расхода топлива)
1. Auto MPG
2. Предсказать расход топлива (mpg)
3. Задания:

- загрузите данные, обработайте пропуски и категориальные признаки;
- обучите модель линейной регрессии, используя в качестве признаков cylinders, horsepower, weight
- рассчитайте MSE и R2;
- визуализируйте зависимость mpg от horsepower с линией регрессии.

Классификация (Диагностика диабета)
1. Pima Indians Diabetes
2. Предсказать наличие диабета (Outcome)
3. Задания:

- загрузите данные, выполните стандартизацию признаков;
- обучите модель логистической регрессии;
- рассчитайте Accuracy, Precision и Recall;
- постройте матрицу ошибок и сделайте выводы о количестве ложноположительных и ложноотрицательных срабатываний.

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.pipeline import make_pipeline

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score


df = pd.read_csv("auto-mpg.csv")


df.columns = df.columns.str.strip()


columns = ["mpg", "cylinders", "horsepower", "weight"]
```

```
df = df[columns].copy()

for column in columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

df = df.dropna(subset=["mpg"])

X = df[["cylinders", "horsepower", "weight"]]
y = df["mpg"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = make_pipeline(
    SimpleImputer(strategy="median"),
    LinearRegression()
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nРезультаты модели:")
print("MSE =", round(mse, 3))
print("R2 =", round(r2, 3))
```

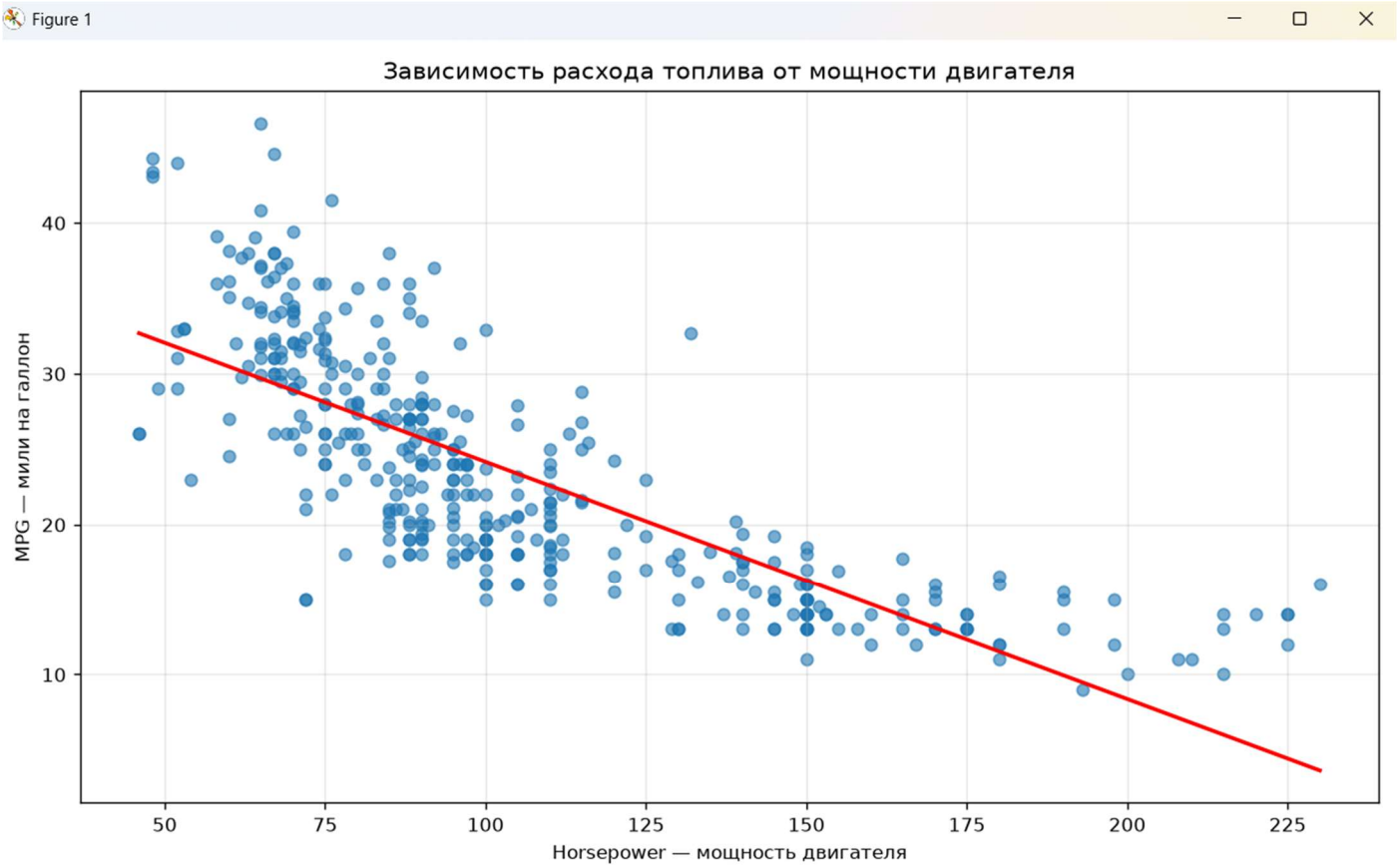
```
results = pd.DataFrame({
    "Реальный mpg": y_test,
    "Предсказанный mpg": y_pred
})

print("\nСравнение реальных и предсказанных значений:")
print(results.head(10).round(2))

plt.figure(figsize=(10, 6))

sns.regplot(
    data=df,
    x="horsepower",
    y="mpg",
    ci=None,
    scatter_kws={"alpha": 0.6},
    line_kws={"color": "red", "linewidth": 2}
)

plt.title("Зависимость расхода топлива от мощности двигателя")
plt.xlabel("Horsepower — мощность двигателя")
plt.ylabel("MPG — мили на галлон")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    confusion_matrix,
    classification_report
)
```

```
df = pd.read_csv("pima-indians-diabetes.csv")

df.columns = df.columns.str.strip()

zero_as_missing = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

for column in zero_as_missing:
    df[column] = df[column].replace(0, float("nan"))

df = df.dropna(subset=["Outcome"])

X = df.drop(columns=["Outcome"])
y = df["Outcome"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
```

```
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test, y_pred, zero_division=0
)

recall = recall_score(
    y_test, y_pred, zero_division=0
)

print("\nМетрики качества модели:")
print("Accuracy =", round(accuracy, 3))
print("Precision =", round(precision, 3))
print("Recall =", round(recall, 3))

cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

print("Матрица ошибок:")
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
```

```
    cmap="Blues",
    xticklabels=["Нет диабета (0)", "Диабет (1)"],
    yticklabels=["Нет диабета (0)", "Диабет (1)"]
)

plt.title("Матрица ошибок — диагностика диабета")
plt.xlabel("Предсказанный класс")
plt.ylabel("Истинный класс")
plt.tight_layout()
plt.show()
```

